

DATA STRUCTURES AND ALGORITHMS

DSA Assignment – AVL Tree Problems

Name : Arunvignesh R S

Roll No: 25BAD012

Question 1

Insert into an AVL Tree while Counting and Removing Duplicate Keys

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int key, h;
    struct Node *l, *r;
} Node;

int h(Node *n){ return n ? n->h : 0; }
int mx(int a,int b){ return a>b?a:b; }

Node* newNode(int k){
    Node*n=malloc(sizeof(Node));
    n->key=k;n->h=1;n->l=n->r=NULL;
    return n;
}

Node* R(Node*y){
    Node*x=y->l,*t=x->r;
    x->r=y;y->l=t;
    y->h=1+mx(h(y->l),h(y->r));
    x->h=1+mx(h(x->l),h(x->r));
    return x;
}

Node* L(Node*x){
    Node*y=x->r,*t=y->l;
    y->l=x;x->r=t;
    x->h=1+mx(h(x->l),h(x->r));
    y->h=1+mx(h(y->l),h(y->r));
    return y;
}

Node* insert(Node*n,int k,int*d){
    if(!n)return newNode(k);
    if(k<n->key)n->l=insert(n->l,k,d);
    else if(k>n->key)n->r=insert(n->r,k,d);
    else{(*d)++;return n;}
    n->h=1+mx(h(n->l),h(n->r));
    int b=h(n->l)-h(n->r);
    if(b>1&&k<n->l->key)return R(n);
    if(b<-1&&k>n->r->key)return L(n);
    if(b>1){n->l=L(n->l);return R(n);}
    if(b<-1){n->r=R(n->r);return L(n);}
    return n;
}

Node* min(Node*n){while(n->l)n=n->l;return n;}

Node* del(Node*n,int k){
    if(!n)return n;
```

```

    if(k<n->key)n->l=del(n->l,k);
    else if(k>n->key)n->r=del(n->r,k);
    else{
        if(!n->l||!n->r){
            Node*t=n->l?n->l:n->r;
            if(!t){free(n);return NULL;}
            *n=*t;free(t);
        }else{
            Node*t=min(n->r);
            n->key=t->key;n->r=del(n->r,t->key);
        }
    }
    n->h=1+mx(h(n->l),h(n->r));
    int b=h(n->l)-h(n->r);
    if(b>1&&h(n->l->l)>=h(n->l->r))return R(n);
    if(b>1){n->l=L(n->l);return R(n);}
    if(b<-1&&h(n->r->r)>=h(n->r->l))return L(n);
    if(b<-1){n->r=R(n->r);return L(n);}
    return n;
}

void in(Node*n){if(n){in(n->l);printf("%d ",n->key);in(n->r);}}

int main(){
    int n,x,m,d,c=0;Node*root=NULL;
    scanf("%d",&n);
    while(n--){scanf("%d",&x);root=insert(root,x,&c);}
    printf("AVL Tree after removing duplicates: ");in(root);
    printf("\nTotal number of duplicates: %d\n",c);
    scanf("%d",&m);
    while(m--){scanf("%d",&d);root=del(root,d);}
    printf("AVL Tree after deletions: ");in(root);
    return 0;
}

```

Question 2

AVL Tree with Associated String Data and Deletion of Minimum Key

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node{
    int key,h;
    char code[32];
    struct Node*l,*r;
}Node;

int h(Node*n){return n?n->h:0;}
int mx(int a,int b){return a>b?a:b;}

Node* newNode(int k,char*c){
    Node*n=malloc(sizeof(Node));
    n->key=k;strcpy(n->code,c);n->h=1;n->l=n->r=NULL;
    return n;
}

Node* R(Node*y){
    Node*x=y->l,*t=x->r;x->r=y;y->l=t;
    y->h=1+mx(h(y->l),h(y->r));x->h=1+mx(h(x->l),h(x->r));
    return x;
}

Node* L(Node*x){
    Node*y=x->r,*t=y->l;y->l=x;x->r=t;
    x->h=1+mx(h(x->l),h(x->r));y->h=1+mx(h(y->l),h(y->r));
    return y;
}

Node* insert(Node*n,int k,char*c){
    if(!n)return newNode(k,c);
    if(k<n->key)n->l=insert(n->l,k,c);
    else if(k>n->key)n->r=insert(n->r,k,c);
    else return n;
    n->h=1+mx(h(n->l),h(n->r));
    int b=h(n->l)-h(n->r);
    if(b>1&&k<n->l->key)return R(n);
    if(b<-1&&k>n->r->key)return L(n);
    if(b>1){n->l=L(n->l);return R(n);}
    if(b<-1){n->r=R(n->r);return L(n);}
    return n;
}

Node* min(Node*n){while(n->l)n=n->l;return n;}

Node* del(Node*n,int k){
    if(!n)return n;
    if(k<n->key)n->l=del(n->l,k);
    else if(k>n->key)n->r=del(n->r,k);
    else{
        if(!n->l||!n->r){
            Node*t=n->l?n->l:n->r;
            if(!t){free(n);return NULL;}
            *n=*t;free(t);
        }else{
            Node*t=min(n->r);
            n->key=t->key;strcpy(n->code,t->code);
            n->r=del(n->r,t->key);
        }
    }
    n->h=1+mx(h(n->l),h(n->r));
    int b=h(n->l)-h(n->r);
    if(b>1&&h(n->l->l)>=h(n->l->r))return R(n);
```

```

    if(b>1){n->l=L(n->l);return R(n);}
    if(b<-1&&h(n->r->r)>=h(n->r->l))return L(n);
    if(b<-1){n->r=R(n->r);return L(n);}
    return n;
}

void in(Node*n){if(n){in(n->l);printf("%d %s\n",n->key,n->code);in(n->r);}}

int main(){
    int n,k;char c[32];Node*root=NULL;
    scanf("%d",&n);
    while(n--){scanf("%d %s",&k,c);root=insert(root,k,c);}
    printf("In-order traversal before deletion:\n");in(root);
    if(root){Node*t=min(root);root=del(root,t->key);}
    printf("In-order traversal after deletion:\n");in(root);
    return 0;
}

```

Question 3

Search for a Key in an AVL Tree

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node{
    int key,h;
    struct Node*l,*r;
}Node;

int h(Node*n){return n?n->h:0;}
int mx(int a,int b){return a>b?a:b;}

Node* newNode(int k){
    Node*n=malloc(sizeof(Node));
    n->key=k;n->h=1;n->l=n->r=NULL;return n;
}

Node* R(Node*y){
    Node*x=y->l,*t=x->r;x->r=y;y->l=t;
    y->h=1+mx(h(y->l),h(y->r));x->h=1+mx(h(x->l),h(x->r));
    return x;
}

Node* L(Node*x){
    Node*y=x->r,*t=y->l;y->l=x;x->r=t;
    x->h=1+mx(h(x->l),h(x->r));y->h=1+mx(h(y->l),h(y->r));
    return y;
}

Node* insert(Node*n,int k){
    if(!n)return newNode(k);
    if(k<n->key)n->l=insert(n->l,k);
    else if(k>n->key)n->r=insert(n->r,k);
    else return n;
    n->h=1+mx(h(n->l),h(n->r));
    int b=h(n->l)-h(n->r);
    if(b>1&& k<n->l->key)return R(n);
    if(b<-1&& k>n->r->key)return L(n);
    if(b>1){n->l=L(n->l);return R(n);}
    if(b<-1){n->r=R(n->r);return L(n);}
    return n;
}

int search(Node*n,int k){
    if(!n)return 0;
    if(n->key==k)return 1;
    return k<n->key?search(n->l,k):search(n->r,k);
}

int main(){
    int x,k;Node*root=NULL;
    while(scanf("%d",&x)==1&&x!=-1)root=insert(root,x);
    scanf("%d",&k);
    if(search(root,k))printf("Key %d found in the AVL tree.\n",k);
    else printf("Key %d not found in the AVL tree.\n",k);
    return 0;
}
```

Question 4

Find the Height of an AVL Tree

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node{
    int key,h;
    struct Node*l,*r;
}Node;

int h(Node*n){return n?n->h:0;}
int mx(int a,int b){return a>b?a:b;}

Node* newNode(int k){
    Node*n=malloc(sizeof(Node));
    n->key=k;n->h=1;n->l=n->r=NULL;return n;
}

Node* R(Node*y){
    Node*x=y->l,*t=x->r;x->r=y;y->l=t;
    y->h=1+mx(h(y->l),h(y->r));x->h=1+mx(h(x->l),h(x->r));
    return x;
}

Node* L(Node*x){
    Node*y=x->r,*t=y->l;y->l=x;x->r=t;
    x->h=1+mx(h(x->l),h(x->r));y->h=1+mx(h(y->l),h(y->r));
    return y;
}

Node* insert(Node*n,int k){
    if(!n)return newNode(k);
    if(k<n->key)n->l=insert(n->l,k);
    else if(k>n->key)n->r=insert(n->r,k);
    else return n;
    n->h=1+mx(h(n->l),h(n->r));
    int b=h(n->l)-h(n->r);
    if(b>1&&k<n->l->key)return R(n);
    if(b<-1&&k>n->r->key)return L(n);
    if(b>1){n->l=L(n->l);return R(n);}
    if(b<-1){n->r=R(n->r);return L(n);}
    return n;
}

int main(){
    int x;Node*root=NULL;
    while(scanf("%d",&x)==1&&x!=-1)root=insert(root,x);
    printf("Height of the AVL tree: %d\n",h(root));
    return 0;
}
```

Question 5

Preorder Traversal of an AVL Tree Before and After Deletion

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node{
    int key,h;
    struct Node*l,*r;
}Node;

int h(Node*n){return n?n->h:0;}
int mx(int a,int b){return a>b?a:b;}

Node* newNode(int k){
    Node*n=malloc(sizeof(Node));
    n->key=k;n->h=1;n->l=n->r=NULL;return n;
}

Node* R(Node*y){
    Node*x=y->l,*t=x->r;x->r=y;y->l=t;
    y->h=1+mx(h(y->l),h(y->r));x->h=1+mx(h(x->l),h(x->r));
    return x;
}

Node* L(Node*x){
    Node*y=x->r,*t=y->l;y->l=x;x->r=t;
    x->h=1+mx(h(x->l),h(x->r));y->h=1+mx(h(y->l),h(y->r));
    return y;
}

Node* insert(Node*n,int k){
    if(!n)return newNode(k);
    if(k<n->key)n->l=insert(n->l,k);
    else if(k>n->key)n->r=insert(n->r,k);
    else return n;
    n->h=1+mx(h(n->l),h(n->r));
    int b=h(n->l)-h(n->r);
    if(b>1&&k<n->l->key)return R(n);
    if(b<-1&&k>n->r->key)return L(n);
    if(b>1){n->l=L(n->l);return R(n);}
    if(b<-1){n->r=R(n->r);return L(n);}
    return n;
}

Node* min(Node*n){while(n->l)n=n->l;return n;}

Node* del(Node*n,int k){
    if(!n)return n;
    if(k<n->key)n->l=del(n->l,k);
    else if(k>n->key)n->r=del(n->r,k);
    else{
        if(!n->l||!n->r){
            Node*t=n->l?n->l:n->r;
            if(!t){free(n);return NULL;}
            *n=*t;free(t);
        }else{
            Node*t=min(n->r);n->key=t->key;n->r=del(n->r,t->key);
        }
    }
    n->h=1+mx(h(n->l),h(n->r));
    int b=h(n->l)-h(n->r);
    if(b>1&&h(n->l->l)>=h(n->l->r))return R(n);
    if(b>1){n->l=L(n->l);return R(n);}
    if(b<-1&&h(n->r->r)>=h(n->r->l))return L(n);
    if(b<-1){n->r=R(n->r);return L(n);}
    return n;
}
```

```

void pre(Node*n){if(n){printf("%d ",n->key);pre(n->l);pre(n->r);}}

int main(){
    int n,x,d;Node*root=NULL;
    scanf("%d",&n);
    while(n--){scanf("%d",&x);root=insert(root,x);}
    scanf("%d",&d);
    printf("Preorder of AVL tree\n");pre(root);
    root=del(root,d);
    printf("\nPreorder after deletion of the element\n");pre(root);
    return 0;
}

```

Question 6

Count Elements Greater Than a Given Value

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node{
    int key,count,h;
    struct Node*l,*r;
}Node;

int h(Node*n){return n?n->h:0;}
int mx(int a,int b){return a>b?a:b;}

Node* newNode(int k){
    Node*n=malloc(sizeof(Node));
    n->key=k;n->count=1;n->h=1;n->l=n->r=NULL;return n;
}

Node* R(Node*y){
    Node*x=y->l,*t=x->r;x->r=y;y->l=t;
    y->h=1+mx(h(y->l),h(y->r));x->h=1+mx(h(x->l),h(x->r));
    return x;
}

Node* L(Node*x){
    Node*y=x->r,*t=y->l;y->l=x;x->r=t;
    x->h=1+mx(h(x->l),h(x->r));y->h=1+mx(h(y->l),h(y->r));
    return y;
}

Node* insert(Node*n,int k){
    if(!n)return newNode(k);
    if(k<n->key)n->l=insert(n->l,k);
    else if(k>n->key)n->r=insert(n->r,k);
    else{n->count++;return n;}
    n->h=1+mx(h(n->l),h(n->r));
    int b=h(n->l)-h(n->r);
    if(b>1&&k<n->l->key)return R(n);
    if(b<-1&&k>n->r->key)return L(n);
    if(b>1){n->l=L(n->l);return R(n);}
    if(b<-1){n->r=R(n->r);return L(n);}
    return n;
}

int greater(Node*n,int x){
    if(!n)return 0;
    if(n->key>x)return n->count+greater(n->l,x)+greater(n->r,x);
    return greater(n->r,x);
}

int main(){
    int n,x;Node*root=NULL;
    scanf("%d",&n);
    while(n--){scanf("%d",&x);root=insert(root,x);}
    scanf("%d",&x);
    printf("Number of elements greater than %d are %d\n",x,greater(root,x));
    return 0;
}
```

GitHub Link

<https://github.com/arunvignesh-07/DSA-24CSI024>